



يونيڤرسيتي سلطان زين العابدين
unisza
UNIVERSITI SULTAN ZAINAL ABIDIN

SUBJECT CODE : CSF35503

SUBJECT NAME : ARTIFICIAL INTELLIGENCE

LECTURER NAME : DR NORHIDAYAH BINTI MUHAMMAD

COURSE NAME : BACHELOR OF SCIENCE COMPUTER
(COMPUTER NETWORK SECURITY) WITH HONOURS

TITLE : CAMPUS ROUTE NAVIGATION SYSTEM

NAME	MATRIC
NUR WASILAH BINTI JAMLIH	078535
NOR AIDA AKMA BINTI ALIAS	078084
NURUL AFIFAH BINTI ABDUL HALIM	081378
FADHILAH BINTI MOHD RUSDI	083214
NIK NUR LIYANA BINTI APANDI	077773

TABLE OF CONTENTS

1.0 INTRODUCTION	3
2.0 PROBLEM DESCRIPTION	4
3.0 SEARCH PROBLEM FORMULATION	5
4.0 ALGORITHM IMPLEMENTATION	10
4.1 Breadth-First Search (BFS)	10
4.2 Depth-First Search (DFS)	12
4.3 Greedy Best-First Search	13
4.4 A* Search	15
4.5 Summary of Algorithm Execution	16
5.0 HEURISTIC DESIGN	17
5.1 Heuristic Function	17
5.2 Suitability of the Heuristic	17
5.3 Admissibility of the Heuristic	18
5.4 Consistency of the Heuristic	18
5.5 Effect of Heuristic on Search Performance	19
6.0 COMPLEXITY ANALYSIS	20
6.1 Complexity Parameters	20
6.2 Breadth-First Search (BFS)	20
6.3 Depth-First Search (DFS)	20
6.4 Greedy Best-First Search	21
6.5 A* Search Algorithm	21
6.6 Algorithm Comparison	22
6.7 Performance Discussion	22
7.0 OPTIMIZATION TECHNIQUES	23
7.1 Reduces Unnecessary Node Exploration	24
7.2 Faster Search Execution	24
7.3 Reduces Memory Usage	24
7.4 Improves Decision Making	24
7.5 Helps Reach the Optimal Path Faster	25
8.0 RESULT COMPARISON	26
8.1 Comparison of Search Algorithms	26
9.0 CONCLUSION	28
10.0 REFERENCES	29

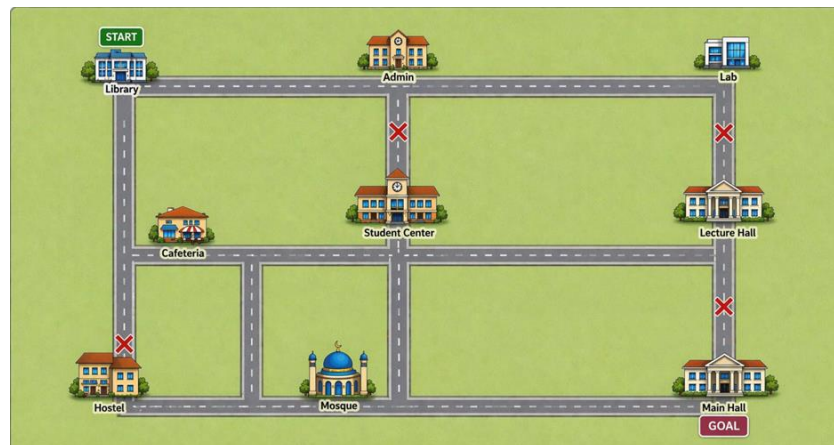
1.0 INTRODUCTION

Artificial Intelligence (AI) plays an important role in solving real-world problems through intelligent decision-making and problem-solving techniques. One of the fundamental approaches in AI is the use of search algorithms, which are used to explore possible solutions and determine the most efficient path to achieve a goal.

The objective of this project was to develop a campus route navigation application that mimics a real-world pathfinding scenario. The task is to determine the optimal route from the library (the starting point) to the Main Hall (the endpoint), while taking blocked roadways into account during the calculation of possible solutions. The campus will be represented as a graph with nodes being the locations of the buildings and the edges being the roadways, which means that the only pathways which can be used will be valid and unblocked.

The methods used in this project to complete the task of finding the optimal route will be the implementation of both uninformed search algorithms (BFS, DFS) and informed search algorithms (Greedy Best-First Search, A* Search), as well as the development of a heuristic function to facilitate more efficient and effective routing algorithms.

2.0 PROBLEM DESCRIPTION



This project deals with the problem of optimizing a path for navigation within a university campus. In this context, there exists an agent who starts from the Library and makes efforts to find the way to the Main Hall as the destination point. Campus is depicted in terms of network that includes buildings which represent nodes while roads connecting them represent edges.

The campus is constituted of nine buildings organized in a grid pattern, namely, Library, Admin, Lab, Cafeteria, Student Center, Lecture Hall, Hostel, Mosque, and Main Hall. The buildings have certain roads that connect them with certain travel distances (or cost). Not all roads, however, can be accessed. There are four roads that have been closed and cannot be traveled by the agent. These include the road from Admin to Student Center, Lab to Lecture Hall, Cafeteria to Hostel, and Lecture Hall to Main Hall. The blocking of roads limits the possible routes and makes it difficult to find the shortest path.

The agent can traverse only the roads which are not blocked or disconnected. The movement is limited to adjacent buildings, and no diagonal movement is allowed. A move would be valid only if there exists a road connecting two buildings and that road is unblocked.

The goal of this problem is to find the optimal path between the Library and Main Hall. Several algorithms including Breadth First Search, Depth First Search, Greedy Best First Search and A* Search are used on the campus graph to find the optimal path between the two locations. Different approaches are followed by these algorithms according to their different strategies including exploration method, use of heuristics, evaluation of costs and efficiency of computing. Therefore, this problem is an appropriate example to analyze uninformed and informed search algorithms in Artificial Intelligence.

3.0 SEARCH PROBLEM FORMULATION

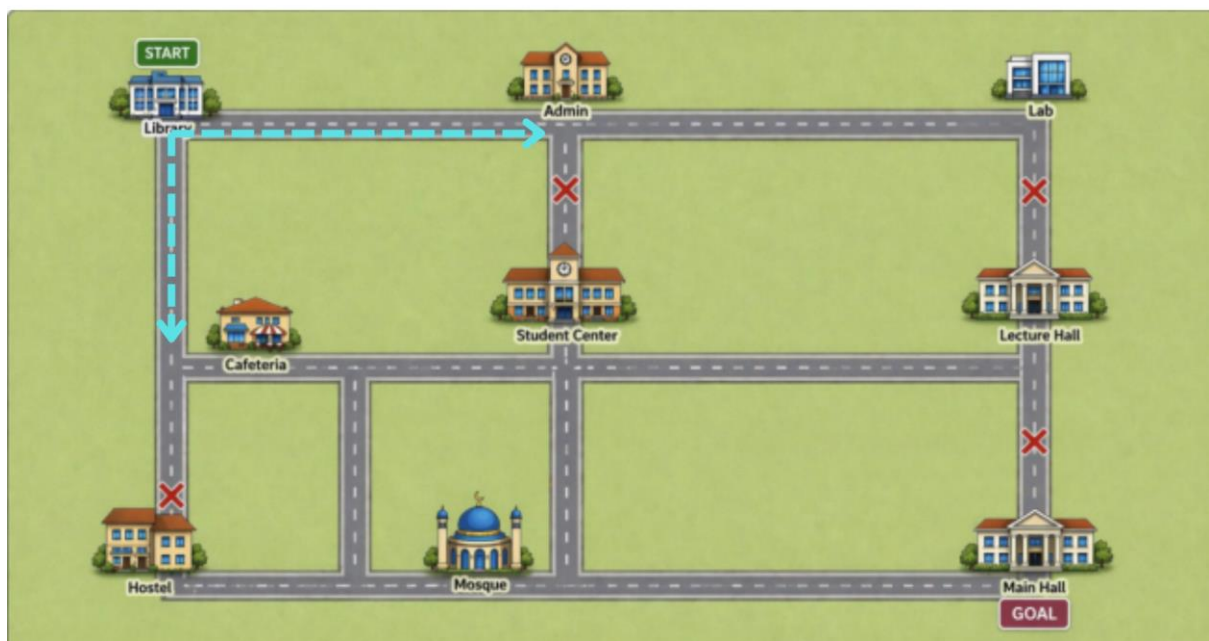
Formulation	Definitions
Initial State	The starting point of the problem where the search begins. It represents the current position before any actions are taken.
Goal State	The desired final destination that the algorithm aims to reach after exploring possible paths.
State Representation	How the problem is modelled in a structured form that can be processed by search algorithms. In this project, the environment is represented as a graph where nodes represent buildings and edges represent valid connections between them.
Operators / Actions	The possible moves that can be applied to a state to reach another state. These actions define how the algorithm transitions from one node to another.
Path Cost	Total cost required to move from the initial state to the goal state. It is calculated by summing the cost of each step taken along the path

Initial State = Library

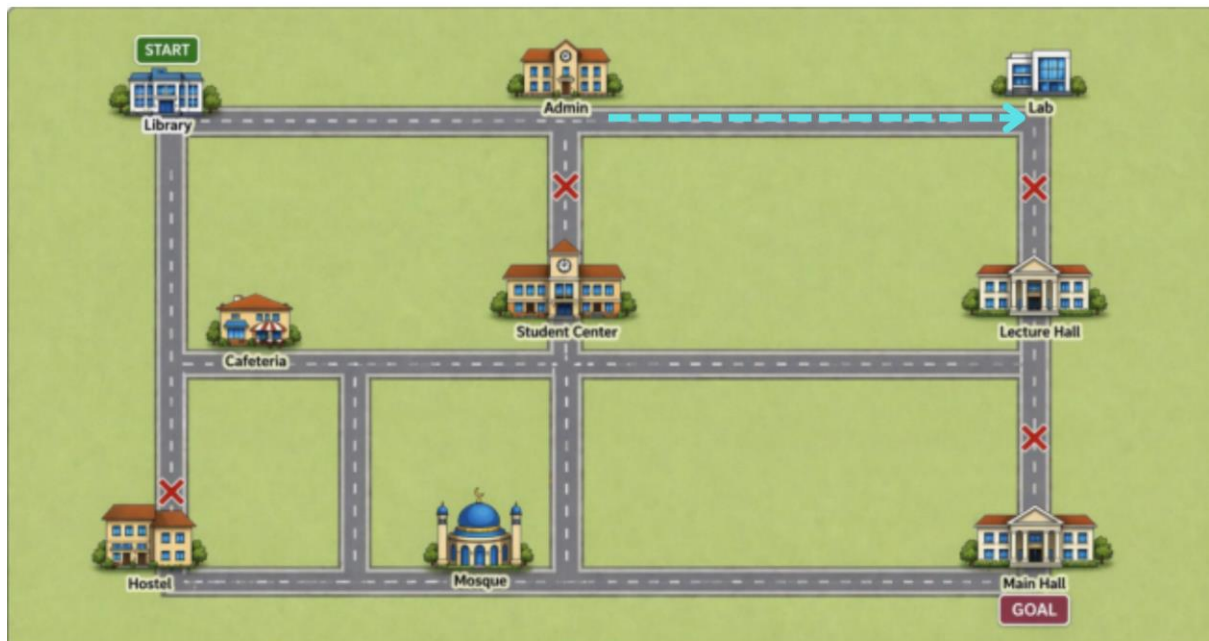
Goal State = Main hall

State Representation:

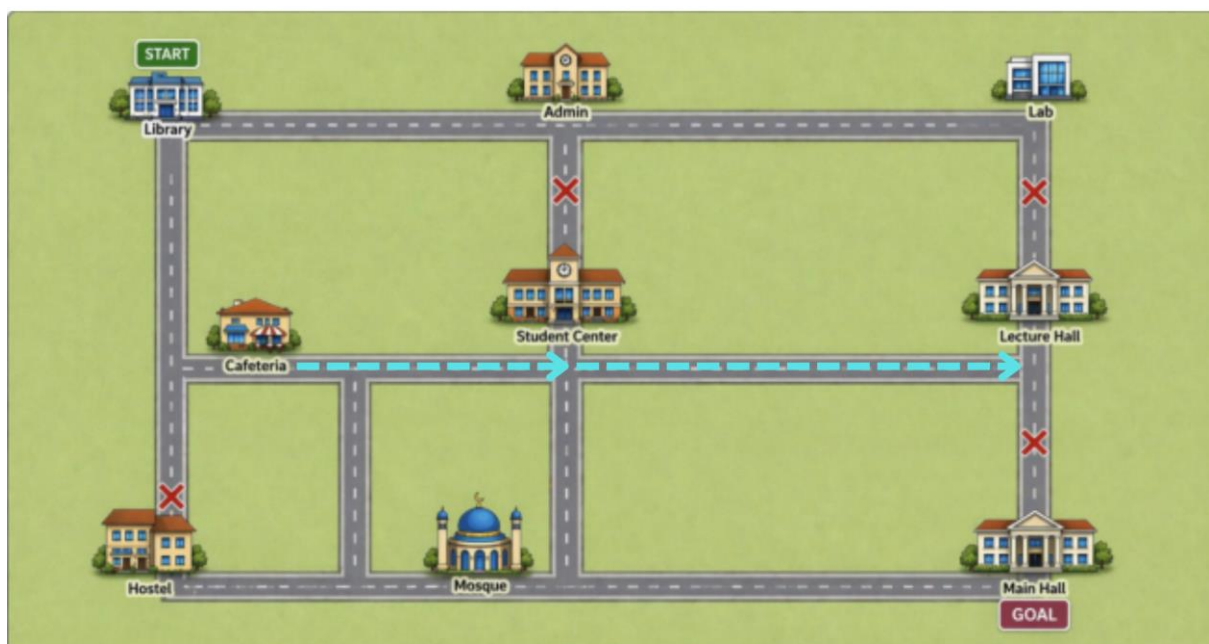
Library to Admin, Cafeteria



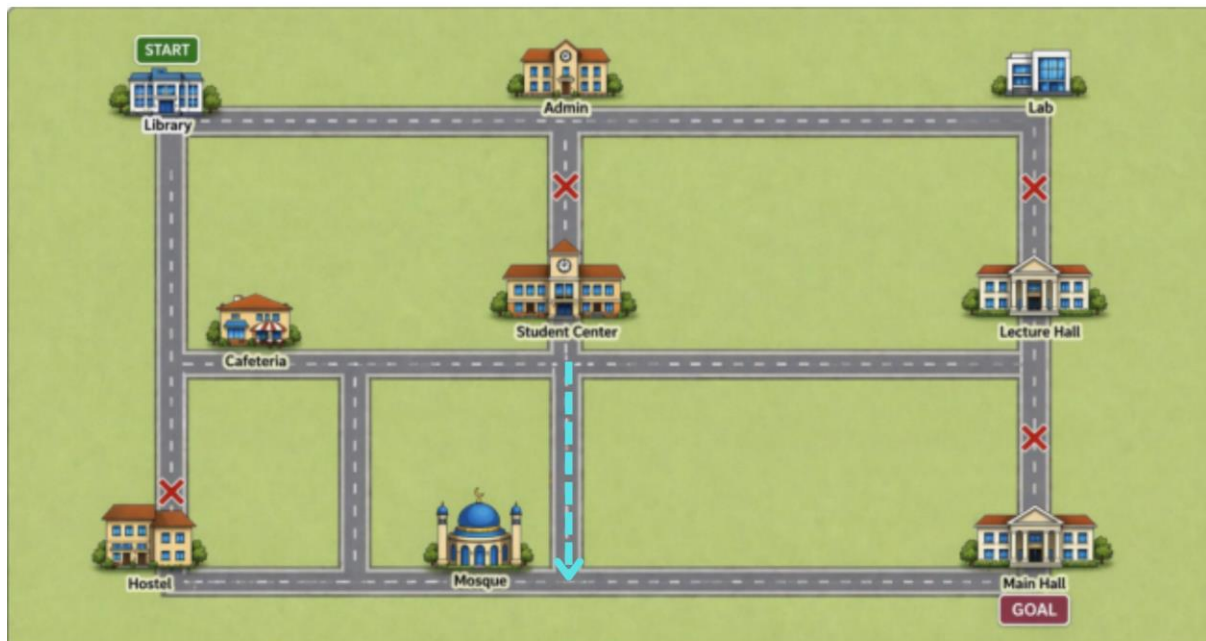
Admin to Lab



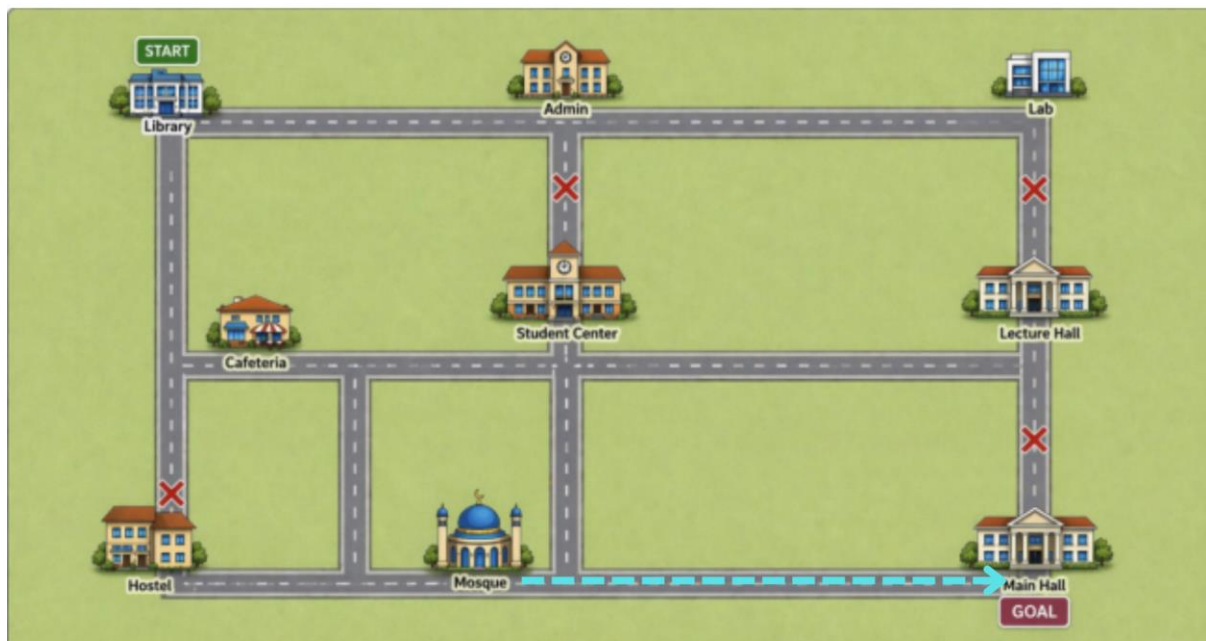
Cafeteria to Student Center, Lecture Hall



Student Center to Mosque

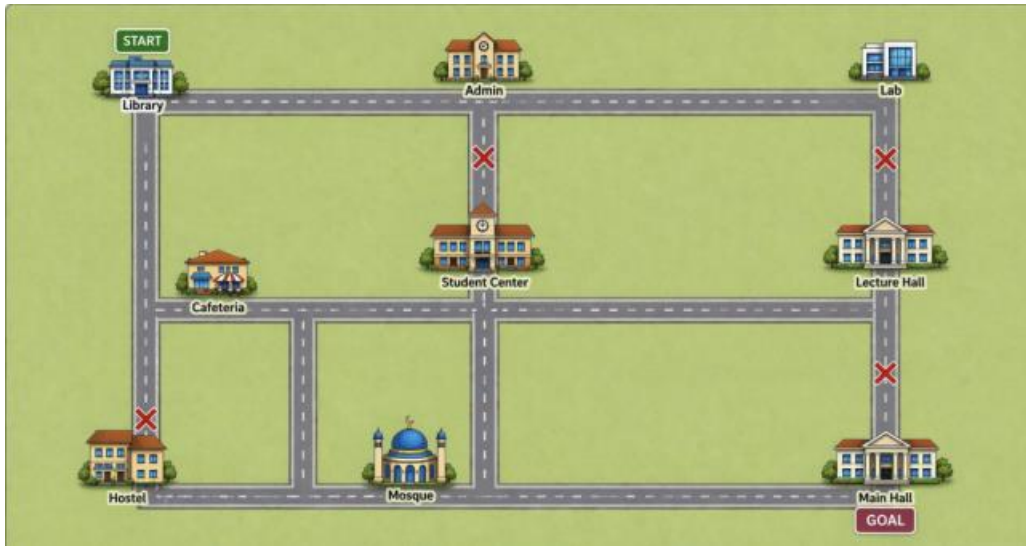


Mosque to Main Hall



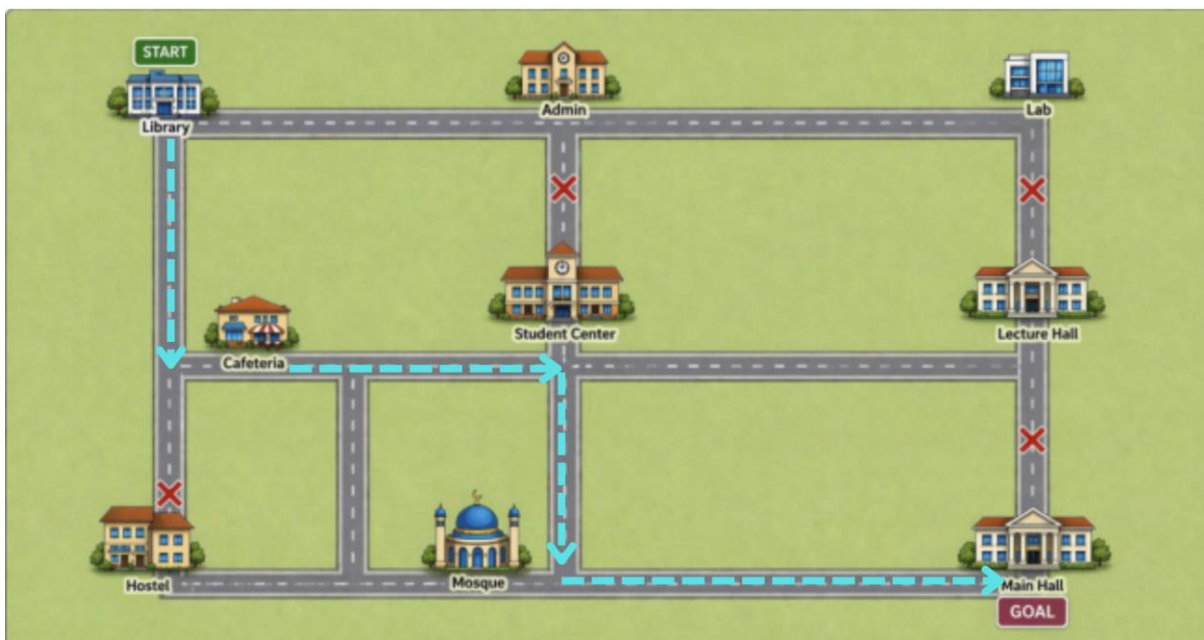
Blocked Connections:

Admin → Student Center
Lab → Lecture Hall
Lecture Hall → Main Hall
Cafeteria → Hostel



Best Valid Path :

Library → Cafeteria → Student Center → Mosque → Main Hall



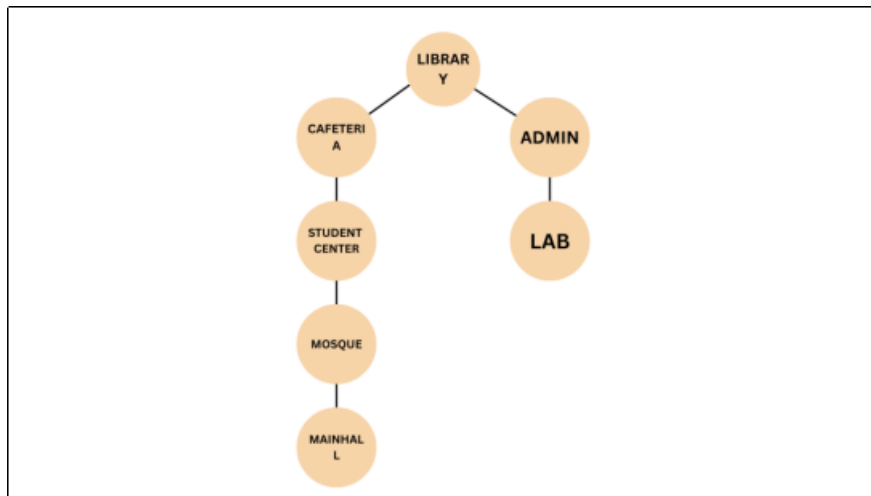
Operators / Actions :

1. At any node, possible actions are:
2. Move Up
3. Move Down
4. Move Left
5. Move Right

BUT only if:

- A road exists
- The road is not blocked

State Representation Diagram



The state space is represented as a tree where the root node is the initial state (Library). Each branch represents possible actions to connected buildings. The path through Cafeteria leads to the goal state (Main Hall), while the path through Admin leads to a dead end due to blocked roads.

4.0 ALGORITHM IMPLEMENTATION

This section discusses the implementation of four different search algorithms for the Campus Route Navigation System. All the algorithms were coded in Python, and the graph for all the algorithms was created based on the diagram in the question. The graph is represented in the form of an adjacency list in which each building is treated as a node and each unblocked road is considered a weighted edge. The four blocked roads are not included in the graph at all, making sure that none of the algorithms pass through them.

Campus graph representation in all four algorithms:

```
GRAPH = {
    'Library':    [('Admin', 4),      ('Cafeteria', 3)],
    'Admin':      [('Library', 4),    ('Lab', 4)],
    'Lab':        [('Admin', 4)],      # dead end - Lab to Lecture Hall is blocked
    'Cafeteria':  [('Library', 3),    ('Student Center', 4)], # Cafeteria to Hostel is blocked
    'Student Center': [('Cafeteria', 4), ('Lecture Hall', 4), ('Mosque', 3)],
    'Lecture Hall': [('Student Center', 4)], # dead end - Lecture Hall to Main Hall is blocked
    'Hostel':     [('Mosque', 4)],      # Cafeteria to Hostel is blocked, only connects to Mosque
    'Mosque':     [('Student Center', 3), ('Hostel', 4), ('Main Hall', 4)],
    'Main Hall':  [('Mosque', 4)],      # this is the GOAL node
}
```

Figure 4.0 Campus Graph Adjacency List

4.1 Breadth-First Search (BFS)

Breadth-First Search is an uninformative search algorithm that explores all nodes at the current depth level before proceeding to the next level using a First-In First-Out (FIFO) queue as its core data structure. The approach guarantees that the first solution found is the shortest path in terms of the number of edges between the starting and goal nodes. However, this feature is a limitation in weighted graphs because BFS does not consider the true cost across edges which could potentially lead to suboptimal cost solutions.

In the implemented code, the algorithm begins at the library node, as defined in the given campus map and systematically expands outward level by level following the available connections between buildings. A visited set is used to avoid revisiting nodes while a parent structure allows reconstruction of the final path.

```

# =====
# ALGORITHM 1: BREADTH-FIRST SEARCH (BFS)
# Type      : Uninformed Search
# Strategy: Explores all nodes level by level using a FIFO queue
# Optimal   : Yes (fewest hops), but not necessarily lowest cost
# Complete: Yes
# Time      :  $O(b^d)$  where b = branching factor, d = depth
# Space     :  $O(b^d)$ 
# =====
def bfs(start, goal):
    print("\n=====")
    print(" BREADTH-FIRST SEARCH (BFS)")
    print("=====")

    queue = deque([start]) # FIFO queue - explore level by level
    visited = {start}      # track visited nodes to avoid revisiting
    parent = {start: None} # track parent of each node for path reconstruction
    expanded = 0           # count how many nodes were expanded

    while queue:
        node = queue.popleft() # remove from front of queue (FIFO)
        expanded += 1
        print(" Visiting:", node)

        # Check if current node is the goal
        if node == goal:
            path = reconstruct_path(parent, start, goal)
            print("\n Goal reached!")
            print(" Path      :", " -> ".join(path))
            print(" Total Cost   :", path_cost(path))
            print(" Nodes Expanded :", expanded)
            return

        # Add all unvisited neighbours to the queue
        for (nb, _) in GRAPH[node]:
            if nb not in visited:
                visited.add(nb)
                parent[nb] = node # record how we reached this neighbour
                queue.append(nb)

```

Figure 4.1.1 Implementation of BFS Algorithm

```

=====
CAMPUS ROUTE NAVIGATION SYSTEM
Start : Library | Goal : Main Hall
=====
Blocked Roads:
- Admin      <--> Student Center
- Lab        <--> Lecture Hall
- Cafeteria  <--> Hostel
- Lecture Hall <--> Main Hall

=====
BREADTH-FIRST SEARCH (BFS)
=====
Visiting: Library
Visiting: Admin
Visiting: Cafeteria
Visiting: Lab
Visiting: Student Center
Visiting: Lecture Hall
Visiting: Mosque
Visiting: Hostel
Visiting: Main Hall

Goal reached!
Path      : Library -> Cafeteria -> Student Center -> Mosque -> Main Hall
Total Cost : 14
Nodes Expanded : 9

```

Figure 4.1.2 Execution Output for BFS

Based on the output, BFS expands all nine nodes before reaching the Main Hall which shows its comprehensive nature. It ensures completeness and guarantees valid solutions, but it also exhibits significant inefficiency. This is because BFS explores many unnecessary nodes without considering direction or cost information. This makes BFS less efficient than informed search algorithms such as A*, especially in larger or more complex environments.

4.2 Depth-First Search (DFS)

DFS is another example of an uninformed search strategy that goes as deep as possible down every branch before reversing its path. LIFO stack forms the basic structure used by DFS algorithm. As opposed to BFS, it cannot be said that DFS will find the shortest or least costly path. On the other hand, it is memory-efficient because it does not store all nodes but just the current path.

Here, the algorithm starts at the Library and dives deep into each branch. In case it hits a dead-end, it moves backward to the previous node and then tries the next branch available.

```
# =====
# ALGORITHM 2: DEPTH-FIRST SEARCH (DFS)
# Type      : Uninformed Search
# Strategy: Explores as deep as possible first using a LIFO stack
# Optimal   : No - does not guarantee shortest or lowest cost path
# Complete  : Yes (for finite graphs with visited set)
# Time      :  $O(b^m)$  where  $m$  = maximum depth
# Space     :  $O(b*m)$  more memory efficient than BFS
# =====
def dfs(start, goal):
    print("\n=====")
    print(" DEPTH-FIRST SEARCH (DFS)")
    print("=====")

    stack = [start]          # LIFO stack - explore deep paths first
    visited = set()          # track visited nodes
    parent = {start: None}   # track parent for path reconstruction
    expanded = 0

    while stack:
        node = stack.pop()    # remove from top of stack (LIFO)

        # Skip if already visited (can happen with stack-based DFS)
        if node in visited:
            continue

        visited.add(node)
        expanded += 1
        print(" Visiting:", node)

        # Check if current node is the goal
        if node == goal:
            path = reconstruct_path(parent, start, goal)
            print("\n Goal reached!")
            print(" Path      :", " -> ".join(path))
            print(" Total Cost   :", path_cost(path))
            print(" Nodes Expanded :", expanded)
            return

        # Add unvisited neighbours to stack in reversed order
        # Reversed so that the first neighbour is explored first
        for (nb, _) in reversed(GRAPH[node]):
            if nb not in visited:
                parent[nb] = node
                stack.append(nb)
```

Figure 4.2.1 Implementation of DFS Algorithm

```

=====
DEPTH-FIRST SEARCH (DFS)
=====
Visiting: Library
Visiting: Admin
Visiting: Lab
Visiting: Cafeteria
Visiting: Student Center
Visiting: Lecture Hall
Visiting: Mosque
Visiting: Hostel
Visiting: Main Hall

Goal reached!
Path      : Library -> Cafeteria -> Student Center -> Mosque -> Main Hall
Total Cost : 14
Nodes Expanded : 9

```

Figure 4.2.2 Execution Output for DFS

DFS first searches the Admin and Lab branches and then moves backward since those routes result in a dead end because there are blocked roads. However, it still discovers the same valid route as BFS using a different search process.

4.3 Greedy Best-First Search

The Greedy Best-First Search is an example of informed search algorithms which make use of the heuristic function $h(n)$, which evaluates the cost from the present node to the target node. It selects the node which looks like being close to the goal according to its estimation without taking into consideration the cost of reaching there. The nodes are kept in a priority queue sorted according to the $h(n)$. The heuristic values used in this approach are the estimates for the straight-line distance from each building to the Main Hall, derived from their positions within the campus layout:

The following heuristic functions are used in the program:

```

HEURISTIC = {
    'Library':      10,    # far from goal, high estimate
    'Admin':        7,
    'Lab':          6,
    'Cafeteria':    9,
    'Student Center': 5,
    'Lecture Hall': 3,
    'Hostel':       8,
    'Mosque':       4,    # one step away from goal
    'Main Hall':    0,    # goal node, zero remaining cost
}

```

```

# =====
# ALGORITHM 3: GREEDY BEST-FIRST SEARCH
# Type : Informed Search
# Strategy: Always expands the node with the lowest h(n) value
# h(n) = estimated distance from node n to goal
# Optimal : No - ignores actual cost g(n), may miss better paths
# Complete: Yes (with visited set)
# Time : O(b^m)
# Space : O(b^m)
# =====
def greedy(start, goal):
    print("\n=====")
    print(" GREEDY BEST-FIRST SEARCH")
    print("=====")

    # Priority queue ordered by heuristic value h(n)
    pq = [(HEURISTIC[start], start)]
    visited = set()
    parent = {start: None}
    expanded = 0

    while pq:
        h, node = heapq.heappop(pq) # pop node with lowest h(n)

        if node in visited:
            continue

        visited.add(node)
        expanded += 1
        print(" Visiting: %-20s h(n) = %d" % (node, h))

        # Check if current node is the goal
        if node == goal:
            path = reconstruct_path(parent, start, goal)
            print("\n Goal reached!")
            print(" Path : ", " -> ".join(path))
            print(" Total Cost : ", path_cost(path))
            print(" Nodes Expanded :", expanded)
            return

        # Add unvisited neighbours to priority queue with their h(n)
        for (nb, _) in GRAPH[node]:
            if nb not in visited:
                parent[nb] = node
                heapq.heappush(pq, (HEURISTIC[nb], nb))

```

Figure 4.3.1 Implementation of Greedy Best-First Algorithm

```

=====
GREEDY BEST-FIRST SEARCH
=====
Visiting: Library          h(n) = 10
Visiting: Admin           h(n) = 7
Visiting: Lab             h(n) = 6
Visiting: Cafeteria       h(n) = 9
Visiting: Student Center  h(n) = 5
Visiting: Lecture Hall    h(n) = 3
Visiting: Mosque          h(n) = 4
Visiting: Main Hall       h(n) = 0

Goal reached!
Path : Library -> Cafeteria -> Student Center -> Mosque -> Main Hall
Total Cost : 14
Nodes Expanded : 8

```

Figure 4.3.2 Execution Output for Greedy Best-First

Greedy algorithm expanded one less node than BFS and DFS due to its heuristic approach. Nonetheless, being based on heuristic values rather than real-life cost, it does not provide the optimum answer every time.

4.4 A* Search

An Informed Search Strategy is known as A* search strategy. It uses a heuristic function $h(n)$ along with the actual cost function $g(n)$ to evaluate the nodes, where the evaluation function will be calculated by adding two functions, which is given by $f(n) = g(n) + h(n)$. It always expands the node having the minimum $f(n)$ value. This strategy is optimal and complete under the condition that $h(n)$ must be admissible.

```
# =====
# ALGORITHM 4: A* SEARCH
# Type      : Informed Search
# Strategy: Expands node with lowest  $f(n) = g(n) + h(n)$ 
#            $g(n)$  = actual cost from start to node n
#            $h(n)$  = estimated cost from node n to goal
# Optimal : Yes - guaranteed if heuristic is ADMISSIBLE
# Complete: Yes
# Time     :  $O(b^d)$ 
# Space    :  $O(b^d)$ 
# Note     : Best algorithm - balances cost and heuristic estimate
# =====
def astar(start, goal):
    print("\n=====")
    print(" A* SEARCH")
    print("=====")

    # Priority queue ordered by  $f(n) = g(n) + h(n)$ 
    pq = [(HEURISTIC[start], 0, start)] # (f, g, node)
    g_cost = {start: 0}                 # actual cost from start to each node
    parent = {start: None}
    expanded = 0

    while pq:
        f, g, node = heapq.heappop(pq) # pop node with lowest f(n)

        # Skip if we already found a cheaper path to this node
        if g > g_cost.get(node, float('inf')):
            continue

        expanded += 1
        print(" Visiting: %-20s g=%-4d h=%-4d f=%d" % (node, g, HEURISTIC[node], f))

        # Check if current node is the goal
        if node == goal:
            path = reconstruct_path(parent, start, goal)
            print("\n Goal reached!")
            print(" Path      :", " -> ".join(path))
            print(" Total Cost   :", path_cost(path))
            print(" Nodes Expanded :", expanded)
            return

        # Explore neighbours and update costs if better path is found
        for (nb, dist) in GRAPH[node]:
            new_g = g + dist # calculate new actual cost to reach neighbour

            # Only update if this path to neighbour is cheaper
            if new_g < g_cost.get(nb, float('inf')):
                g_cost[nb] = new_g
                parent[nb] = node
                f_val = new_g + HEURISTIC[nb] #  $f(n) = g(n) + h(n)$ 
                heapq.heappush(pq, (f_val, new_g, nb))
```

Figure 4.4.1 Implementation of A* Search Algorithm

```

=====
A* SEARCH
=====
Visiting: Library          g=0    h=10   f=10
Visiting: Admin            g=4    h=7    f=11
Visiting: Cafeteria        g=3    h=9    f=12
Visiting: Student Center   g=7    h=5    f=12
Visiting: Lab              g=8    h=6    f=14
Visiting: Mosque           g=10   h=4    f=14
Visiting: Lecture Hall     g=11   h=3    f=14
Visiting: Main Hall        g=14   h=0    f=14

Goal reached!
Path      : Library -> Cafeteria -> Student Center -> Mosque -> Main Hall
Total Cost : 14
Nodes Expanded : 8

```

Figure 4.4.2 Execution Output for A* Search

A* generated an optimal route like the others, but using heuristic data to avoid unnecessary searching. It is assured to yield the most economical route as all its heuristics are admissible; that is, they do not exceed the exact distance to Main Hall.

4.5 Summary of Algorithm Execution

All four algorithms correctly identified the optimal path as Library -> Cafeteria -> Student Center -> Mosque -> Main Hall, with an aggregate travel cost of 14 units. This is the sole possible path within the graph, given that there are four closed paths. The following table gives a summary of how well the algorithms performed during their execution.

```

=====
ALGORITHM COMPARISON TABLE
=====
Algorithm  Path                                     Cost  Nodes Expanded
-----
BFS        Library->Cafeteria->Student Center->Mosque->Main Hall 14      9
DFS        Library->Cafeteria->Student Center->Mosque->Main Hall 14      9
Greedy     Library->Cafeteria->Student Center->Mosque->Main Hall 14      8
A*         Library->Cafeteria->Student Center->Mosque->Main Hall 14      8
=====
Best Algorithm : A* Search
Reason        : Optimal path guaranteed with
                  fewest nodes expanded
=====

```

Figure 4.5.1 Algorithm Comparison Table

The informed algorithms (Greedy and A* algorithms) proved to be efficient because of their use of heuristic information. Hence, fewer nodes were expanded as compared to the two non-intelligent algorithms (BFS and DFS), which searched the entire graph without the use of any directions.

5.0 HEURISTIC DESIGN

5.1 Heuristic Function

The heuristic function for the Campus Route Navigation System is one which calculates the distance between the present location of the person and his destination which is the Main Hall.

The heuristic used is:

$$h(n) = \text{Estimated distance from current node to Main Hall}$$

The heuristic values are assigned based on the estimated number of movement steps between buildings.

Building	Heuristic Value $h(n)$
Main Hall	0
Lecture Hall	1
Student Center	2
Cafeteria	2
Admin	3
Lab	4
Mosque	4
Library	5
Hostel	6

Heuristic Table

Buildings with lower heuristic values are considered closer to the Main Hall.

5.2 Suitability of the Heuristic

This is an appropriate heuristic since it measures how close a node is to its goal state. With the heuristic, informed searching algorithms like Greedy Best-First and A* Search become more efficient as they work their way to the Main Hall.

The heuristic improves search performance by:

- Reducing unnecessary node exploration
- Guiding the search toward the goal
- Reducing search time
- Improving decision making during traversal

For example, if the algorithm compares:

Lecture Hall $\rightarrow h(n)=1$

Hostel $\rightarrow h(n)=6$

The algorithm will prioritize Lecture Hall because it is estimated to be closer to the Main Hall.

5.3 Admissibility of the Heuristic

A heuristic is admissible if it never overestimates the actual shortest path cost to the goal.

The admissibility condition is:

$$h(n) \leq h^*(n)$$

where:

- $h(n)$ = heuristic estimate
- $h^*(n)$ = actual shortest path cost

Heuristic is applicable since it determines the proximity of a node to the desired end state. As such, it assists the informed searching techniques, including Greedy Best-First Search and A* Search, to locate the Main Hall efficiently.

Example:

Since the minimum path between Library and Main Hall takes 6 steps, but the estimated cost using the heuristic is 5:

$$5 \leq 6$$

An admissible heuristic always underestimates the real shortest path cost to the goal.

5.4 Consistency of the Heuristic

A heuristic is consistent if:

$$h(n) \leq c(n, n') + h(n')$$

Where:

- $c(n, n')$ = movement cost between two connected nodes.

The heuristic is consistent because the heuristic values decrease gradually when moving closer to the Main Hall.

Example:

From Library to Admin:

- $h(\text{Library}) = 4$
- Cost = 1

- $H(\text{Admin}) = 3$

Check consistency:

$$4 \leq 1 + 3$$

$$4 \leq 4$$

Thus, the heuristics are consistent.

5.5 Effect of Heuristic on Search Performance

The heuristic significantly improves informed search algorithms.

Without Heuristic	With Heuristic
Explores many unnecessary nodes	Focuses toward goal
Higher execution time	Faster search
No direction guidance	Goal-oriented traversal
More memory usage	Reduced code expansion

In A* Search, the evaluation function used is:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ = actual cost from the start node
- $h(n)$ = estimated cost to the goal node

This allows A* Search to find the optimal path efficiently.

6.0 COMPLEXITY ANALYSIS

6.1 Complexity Parameters

Symbol	Meaning
b	Branching factor
d	Depth of shallowest solution
m	Maximum depth of state space

6.2 Breadth-First Search (BFS)

Time Complexity

$$O(b^d)$$

BFS explores all nodes level by level until the goal is found.

Space Complexity

$$O(b^d)$$

BFS stores all generated nodes in memory.

Analysis

- Complete: Yes
- Optimal: Yes
- Memory Usage: High

6.3 Depth-First Search (DFS)

Time Complexity

$$O(b^m)$$

DFS may explore deep paths before finding the goal.

Space Complexity

$$O(b^m)$$

DFS stores only the current traversal path.

Analysis

- Complete: No

- Optimal: No
- Memory Usage: Low

6.4 Greedy Best-First Search

Greedy Search selects the node with the lowest heuristic value.

Time Complexity

$O(b^m)$

Depends on the heuristic quality.

Space Complexity

$O(b^m)$

Stores generated nodes in memory.

Analysis

- Faster with good heuristic
- Not guaranteed optimal
- Can choose wrong paths

6.5 A* Search Algorithm

A* Search combines path cost and heuristic estimate.

Time Complexity

$O(b^d)$

Space Complexity

$O(b^d)$

Analysis

- Complete: Yes
- Optimal: Yes
- Efficient search performance
- High memory usage

6.6 Algorithm Comparison

Algorithm	Time Complexity	Space Complexity	Complete	Optimal
BFS	$O(b^d)$	$O(b^d)$	Yes	Yes
DFS	$O(b^m)$	$O(b^m)$	No	No
Greedy Best-First Search	$O(b^m)$	$O(b^m)$	Depends	No
A* Search	$O(b^d)$	$O(b^d)$	Yes	Yes

6.7 Performance Discussion

- Breadth-First Search (BFS)

BFS is assured to get to the target by using the shortest route available, but it uses a lot of memory because all the nodes visited must be held in memory constantly.

- Depth-First Search (DFS)

DFS is a search technique that needs less memory and can dig very deeply into a search tree for a long time without returning any result. In addition, the depth-first search technique does not ensure the shortest path.

- Greedy Best First Search

Most efficient when speed is taken into consideration, but there is no guarantee that an optimum solution is obtained.

- A* Search Technique

It evaluates the cost of traveling along the path visited so far versus the heuristic estimate of cost from current state to goal state.

7.0 OPTIMIZATION TECHNIQUES

The project uses a technique to make it work better which is called the Improved Heuristic Technique. This technique is used in some search algorithms like Greedy Best-First Search and A* Search. The Improved Heuristic Technique has a function that helps the algorithm figure out how close it is to the Main Hall.

When the algorithm has an idea of how close it is to the Main Hall it can make better choices and not waste time looking in the wrong places. The original way of doing things gave values based on how many steps it would take to get between buildings. The Improved Heuristic Technique can be made even better by using a more accurate way to measure the distance between nodes and the Main Hall. This way the algorithm can pick the nodes that're close to the Main Hall, which is the destination.

The improved heuristic function used is:

$h(n)$ = Estimated shortest distance from current node to Main Hall

The heuristic values are designed to always underestimate or equal the real shortest path cost. Therefore, the heuristic remains admissible and suitable for A* Search.

Example of Improved heuristic values:

Building	Improved Heuristics $h(n)$
Main Hall	0
Mosque	1
Student Center	2
Cafeteria	3
Library	4
Admin	5
Lab	6
Hostel	4

The values are obtained by counting the estimated number of valid movement steps needed to reach the Main Hall. For example:

- Mosque → Main Hall = 1 step
- Student Center → Mosque → Main Hall = 2 steps

- Library → Cafeteria → Student Center → Mosque → Main Hall = 4 steps

Blocked roads do not count in the calculation because they cannot be used during traversal. Therefore, the heuristic becomes more realistic and accurate.

The improved heuristic enhances algorithm performance in several ways.

7.1 Reduces Unnecessary Node Exploration

If the heuristic is not accurate enough, the algorithm may traverse numerous irrelevant nodes before finding the solution. The better heuristic will lead the algorithm straight to the nodes expected to be nearer to the Main Hall.

For example:

- Student Center → $h(n) = 2$
- Lab → $h(n) = 6$

The algorithm will prioritize Student Center because it is predicted to be closer to the goal. This reduces exploration of less useful paths.

7.2 Faster Search Execution

Instead of exploring nodes in all directions, an improved heuristic provides a "direction guidance" that focuses the search toward the destination. The search process becomes faster compared to uninformed algorithms like BFS and DFS, which explore nodes without direction.

7.3 Reduces Memory Usage

An effective heuristic reduces the "node expansion" or the number of nodes that need to be stored in memory during the search process.

7.4 Improves Decision Making

The algorithm can prioritize paths that are mathematically estimated to be more promising (e.g., prioritizing a node with a heuristic value of 1 over a node with a value of 6). This helps Greedy Best-First Search and A* Search make better decisions during traversal.

In A* Search, the evaluation function is:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ = actual cost from the start node
- $h(n)$ = estimated cost to the goal

A more accurate $h(n)$ allows A* Search to calculate better $f(n)$ values, resulting in more efficient path selection.

7.5 Helps Reach the Optimal Path Faster

The improved heuristic focuses the search toward the valid shortest route:

Library → Cafeteria → Student Center → Mosque → Main Hall

As a result, the algorithms can reach the destination faster while avoiding blocked or unnecessary paths.

8.0 RESULT COMPARISON

The four search algorithms used in this project were able to find a valid route from the Library to the Main Hall and avoid blocked roads in the campus graph successfully. All algorithms gave the same final path, but they differed in their search strategy, how they expanded nodes, memory usage, how fast they ran, and how optimal they were.

8.1 Comparison of Search Algorithms

Algorithm	Type	Path Found	Total Cost	Nodes Expanded	Complete	Optimal	Memory Usage	Overall Performance
Breadth-First Search (BFS)	Uninformed Search	Library → Cafeteria → Student Center → Mosque → Main Hall	14	9	Yes	Yes	High	Accurate but less efficient due to large node expansion
Depth-First Search (DFS)	Uninformed Search	Library → Cafeteria → Student Center → Mosque → Main Hall	14	9	No	No	Low	Memory efficient but may explore dead-end paths
Greedy Best-First Search	Informed Search	Library → Cafeteria → Student Center → Mosque → Main Hall	14	8	Depends on heuristic	No	Moderate	Faster search using heuristic guidance
A* Search	Informed Search	Library → Cafeteria → Student Center → Mosque → Main Hall	14	8	Yes	Yes	High	Most efficient and optimal algorithm

The comparison table indicates that all algorithms were able to reach the Main Hall and obtain the same valid path of total cost 14. The algorithms were very different with respect to their efficiency and search behaviour, however.

Breadth-First Search (BFS) is because it would visit the graph level by level and would give the best solution. But BFS added too many of the unnecessary nodes and consumed the lots of memory as all the nodes created needed to be kept in memory during traversal.

Depth-First Search (DFS) used less memory because it would explore one branch of the tree in depth and backtrack. However, DFS started dead ends first and in more complicated cases it was not able to assure the shortest path or the best route.

Another heuristic was used by Greedy Best-First Search to estimate which nodes could be closer to the Main Hall to improve performance. This helped save on unnecessary exploration and speed up execution. Greedy Search, however, only used its heuristic estimates, and did not take into account actual path costs so it is not always optimal.

A* Search achieved the optimum overall performance amongst all algorithms, taking into account both accuracy and speed of search due to the use of actual path cost $g(n)$ and heuristic estimate $h(n)$. This algorithm expanded fewer nodes than BFS and DFS, but still ensured the best path. Thus, A* search is the most appropriate algorithm for a Campus Route Navigation System.

9.0 CONCLUSION

Finally, this project has successfully illustrated how Artificial Intelligence search algorithms can be used to solve a real world's problem of navigating a campus. The campus environment was represented as a graph of nodes (buildings) and edges (roads) with some of the roads being blocked, thereby making the search process more complex.

The four search algorithms (Breadth-First Search (BFS), Depth-First Search (DFS), Greedy Best-First Search and A* Search) were developed and tested. The results indicated that for all algorithms there was always a valid route from the Library to the Main Hall. But every algorithm exhibited different characteristics when it came to node expansion, memory usage, search efficiency, and optimality.

Without any heuristic guidance, uninformed search algorithms BFS & DFS traversed the graph in an unproductive manner, leading to unnecessary exploration of nodes and inefficiency. However, the informed search algorithms Greedy Best-First Search and A* Search performed well with heuristic information to facilitate the search process to move closer towards the goal state, and of all the search algorithms, A* Search performed best, by combining both path cost and heuristic estimation to ensure the optimal solution is found efficiently.

What's more, the heuristic design and optimization methods enhanced the search efficiency by cutting down on redundant exploring, optimizing decision making and assisting the algorithms to reach the goal faster. This project also enhanced the knowledge of the formulation of AI problems, heuristic evaluation, complexity analysis, and implementation of search algorithms in Python.

The project was overall successfully completed, showcasing the potential of Artificial Intelligence skills to efficiently address real-world problems related to navigation and pathfinding.

10.0 REFERENCES

1. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
2. Poole, D. L., & Mackworth, A. K. (2017). *Artificial Intelligence: Foundations of Computational Agents* (2nd ed.). Cambridge University Press.
3. Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.
4. Putra, D., Andhika Nur Maulana, Shasha Billa Febrianti, Nabila Camelia, Sabastian Kaka Hutagalung, & Ahsanun Naseh Khudori. (2025). Comparison of Breadth-First Search (BFS) and Depth-First Search (DFS) Algorithms for Shortest Search in Campus Labyrinth. *Deleted Journal*, 2(2), 43–51.
<https://doi.org/10.47794/jesica.v2i2.14>
5. Sakaue, S., & Oki, T. (2022). Sample Complexity of Learning Heuristic Functions for Greedy-Best-First and A* Search. *ArXiv.org*. <https://arxiv.org/abs/2205.09963>
6. Shaukat, F. S., Shafique, A., & Qadri, A. I. (2022). Comparative Analysis of Search Algorithms in AI. *ResearchGate*. <https://doi.org/10.13140/RG.2.2.29282.61123>
7. *State Space Search in Artificial Intelligence*. (n.d.). AlmaBetter.
<https://www.almabetter.com/bytes/tutorials/artificial-intelligence/state-space-search-in-artificial-intelligence>
8. Moustakas, C. (1990). *Heuristic research: Design, methodology, and applications*. Sage Publications.
https://books.google.com.my/books?hl=en&lr=&id=y3HwBQAAQBAJ&oi=fnd&pg=PP1&dq=heuristic+design&ots=QIRUTjW7J7&sig=9xB-aJ1Pa7mBcoJEtZxNd00kDjs&redir_esc=y#v=onepage&q=heuristic%20design&f=false
9. Bellomo, N., Bianca, C., & Delitala, M. (2009). Complexity analysis and mathematical tools towards the modelling of living systems. *Physics of Life Reviews*, 6(3), 144-175.
<https://www.sciencedirect.com/science/article/abs/pii/S1571064509000232>